

Отчёт по лабораторной работе №8

Математические основы защиты информации и информационной безопасности

Ли Хан

Содержание

1	Цель работы	5
2	Подробное описание алгоритмов	6
2.1	Сложение неотрицательных целых чисел	6
2.1.1	Шаги реализации:	6
2.1.2	компилятора	7
2.1.3	результат	8
2.2	Вычитание неотрицательных целых чисел	8
2.2.1	Шаги реализации:	8
2.2.2	компилятора	9
2.2.3	результат	10
2.3	Умножение столбиком	10
2.3.1	Шаги реализации:	10
2.3.2	компилятора	12
2.3.3	результат	13
2.4	Быстрый столбик	13
2.4.1	Шаги реализации:	13
2.4.2	компилятора	14
2.4.3	результат	14
2.5	Деление многоразрядных чисел	15
2.5.1	Шаги реализации:	15
2.5.2	компилятора	16
2.5.3	результат	17
3	Вывод	18

Список иллюстраций

2.1	Сложение неотрицательных целых чисел компилятора	7
2.2	Сложение неотрицательных целых чисел результат	8
2.3	Вычитание неотрицательных целых чисел компилятора	9
2.4	Вычитание неотрицательных целых чисел результат	10
2.5	Умножение столбиком компилятора	12
2.6	Умножение столбиком результат	13
2.7	Быстрый столбик компилятора	14
2.8	Быстрый столбик результат	14
2.9	Деление многоразрядных чисел компилятора	16
2.10	Деление многоразрядных чисел результат	17

Список таблиц

1 Цель работы

Изучить и реализовать основные алгоритмы целочисленной арифметики многократной точности. Целью работы является освоение методов выполнения арифметических операций (сложение, вычитание, умножение и деление) над числами, разрядность которых превышает стандартные типы данных ЭВМ, с использованием позиционной системы счисления с основанием b .

2 Подробное описание алгоритмов

2.1 Сложение неотрицательных целых чисел

Алгоритм имитирует ручное сложение столбиком. Процесс начинается с младших разрядов и включает перенос k в старшие разряды.

2.1.1 Шаги реализации:

- Шаг 1: Установка индекса $j = n$ и начального переноса $k = 0$.
- Шаг 2: Вычисление текущего разряда суммы как $(u_j + v_j + k) \pmod{b}$ и обновление переноса k как целой части от деления суммы на b .
- Шаг 3: Переход к более старшему разряду до завершения всей длины числа.

2.1.2 компилятора

```
def big_int_add(u, v, b=10):  
    """  
    实现两个大整数的加法。  
    Реализация сложения двух больших целых чисел.  
    """  
  
    # 确定最大位数，并将较短的数在高位补0以对齐  
    # Определяем максимальную разрядность и дополня  
    n = max(len(u), len(v))  
    u_ptr = [0] * (n - len(u)) + list(u)  
    v_ptr = [0] * (n - len(v)) + list(v)  
  
    # 初始化变量: j 为当前位索引, k 为进位 (0 或 1)  
    # Инициализация: j — индекс разряда, k — перенос (  
    j = n - 1  
    k = 0  
    # w 存储结果，长度为 n+1 以存储可能的最高位进位  
    # w хранит результат, длина n+1 для возможного пер  
    w = [0] * (n + 1)  
  
    # 从最低位到最高位进行循环计算  
    # Цикл вычислений от младшего разряда к старшем!  
    while j >= 0:  
        # 当前位的总和 = 两个加数的当前位 + 前一位的进位  
        # Сумма текущего разряда = сумма разрядов сл  
        current_sum = u_ptr[j] + v_ptr[j] + k  
  
        # 将当前位的结果（对基数取模）存入结果数组  
        # Записываем результат разряда (остаток от д  
        w[j + 1] = current_sum % b  
  
        # 计算新的进位（总和除以基数的整数部分）  
        # Вычисляем новый перенос (целая часть от де  
        k = current_sum // b  
  
        # 索引向高位移动  
        # Переход к следующему старшему разряду  
        j -= 1  
  
    # 将最终的进位赋值给结果数组的第0位  
    # Присваиваем последний перенос в самый старший  
    w[0] = k  
  
    # 样例 (Пример): 998 + 7 = 1005  
    return w if w[0] != 0 else w[1:]
```

Рис. 2.1: Сложение неотрицательных целых чисел компилятора

2.1.3 результат

```
# 运行示例:  
print(f"加法样例 (998 + 7): {big_int_add([9, 9, 8], [7])}")
```

加法样例 (998 + 7): [1, 0, 0, 5]

Рис. 2.2: Сложение неотрицательных целых чисел результат

2.2 Вычитание неотрицательных целых чисел

Алгоритм реализует операцию $u - v$ при условии $u > v$. Переменная k используется для фиксации займа из старшего разряда.

2.2.1 Шаги реализации:

- Шаг 1: Инициализация индекса и переменной займа k .
- Шаг 2: Вычисление разности разрядов с учетом займа и обновление k .
- Шаг 3: Циклическое повторение для всех разрядов.

2.2.2 компилятора

```
import math

def big_int_subtract(u, v, b=10):
    """
    实现两个大整数的减法 (u - v)。
    Реализация вычитания двух больших ч
    """
    n = len(u)
    # 将减数 v 在高位补0以对齐被减数 u 的长度
    # Дополняем вычитаемое v нулями слева
    v_ptr = [0] * (n - len(v)) + list(v)

    # k 为借位变量，通常为 0 或 -1
    # k — переменная заема, обычно 0 или
    j = n - 1
    k = 0
    w = [0] * n

    # 从最低位到最高位依次执行减法
    # Поразрядное вычитание от младших
    while j >= 0:
        # 当前位的差值 = u的当前位 - v的当前位 + 借位
        # Разность разряда = разряд u - p
        current_diff = u[j] - v_ptr[j] + k

        # 存储计算结果，通过取模确保结果在 [0, b-1] 范围内
        # Сохраняем результат, mod b гаран
        w[j] = current_diff % b

        # 更新借位：如果差值为负，则需要向高位借 1 (k = -1)
        # Обновляем заем: если разность
        k = math.floor(current_diff / b)

        j -= 1

    # 样例 (Пример): 1000 - 1 = 999
    return w
```

Рис. 2.3: Вычитание неотрицательных целых чисел компилятора

2.2.3 результат

```
# 运行示例:  
print(f"减法样例 (1000 - 1): {big_int_subtract([1, 0, 0, 0], [1])}")
```

减法样例 (1000 - 1): [0, 9, 9, 9]

Рис. 2.4: Вычитание неотрицательных целых чисел результат

2.3 Умножение столбиком

Классический метод умножения, где каждый разряд множителя v последовательно умножается на все разряды множимого u .

2.3.1 Шаги реализации:

- Шаг 1: Обнуление массива результата w .
- Шаг 2: Пропуск итерации, если текущий разряд множителя равен нулю.
- Шаг 3: Вычисление промежуточных произведений t , их суммирование с текущим значением разряда w и обработка переноса k .
- Шаг 4: Повторение для всех разрядов множителя.

2.3.2 компилятора

```
def big_int_multiply_standard(u, v, b=10):  
    """  
    经典列乘法实现。  
    Реализация классического умножения стол  
    """  
  
    n, m = len(u), len(v)  
    # 结果数组初始化为0，长度最大为 n + m  
    # Массив результата длиной n + m, заполнен  
    w = [0] * (m + n)  
  
    # 外层循环：遍历乘数 v 的每一位（从低到高）  
    # Внешний цикл: проходим по разрядам мно  
    for j in range(m - 1, -1, -1):  
        # 如果当前乘数位为 0，则该位产生的乘积全为 0，跳过计算  
        # Если разряд множителя v_j равен 0, пр  
        if v[j] == 0:  
            continue  
  
        i = n - 1  
        k = 0 # 乘法过程中产生的进位 (перенос)  
  
        # 内层循环：遍历被乘数 u 的每一位  
        # Внутренний цикл: проходим по разря  
        while i >= 0:  
            # 计算临时乘积 = u_i * v_j + 结果位已有的值 + 进位  
            # t = u_i * v_j + Текущее значение в w  
            t = u[i] * v[j] + w[i + j + 1] + k  
  
            # 更新结果位的值  
            # Обновляем значение текущего ра  
            w[i + j + 1] = t % b  
  
            # 计算向高位传递的进位  
            # Вычисляем новый перенос k  
            k = t // b  
            i -= 1  
  
        # 存储该轮乘法在最高位产生的进位  
        # Сохраняем последний перенос этой и  
        w[j] = k  
  
    # 样例 (Пример): 12 * 13 = 156  
    return w
```

Рис. 2.5: Умножение столбиком компилятора

2.3.3 результат

```
# 运行示例:  
print(f"列乘法样例 (12 * 13): {big_int_multiply_standard([1, 2], [1, 3])}")  
  
列乘法样例 (12 * 13): [0, 1, 5, 6]
```

Рис. 2.6: Умножение столбиком результат

2.4 Быстрый столбик

Оптимизированный алгоритм умножения, использующий один аккумулятор t для вычисления суммы вкладов всех пар разрядов в конкретную позицию результата.

2.4.1 Шаги реализации:

- Шаг 1: Обнуление аккумулятора t .
- Шаг 2: Вычисление суммы произведений $u_i \cdot v_j$ для каждой позиции s результата.
- Шаг 3: Сохранение результата разряда и обновление значения аккумулятора для следующего шага.

2.4.2 компилятора

```
def big_int_multiply_fast(u, v, b=10):
    """
    快速列乘法实现，使用累加器统一处理。
    Оптимизированное умножение с использованием
    """
    n, m = len(u), len(v)
    w = [0] * (m + n)
    t = 0 # 累加器 (аккумулятор)

    # 遍历结果数组的所有位位置 s
    # Проходим по всем позициям разрядов резу
    for s in range(m + n):
        # 寻找所有满足下标条件的 u_i * v_j 对并累加到 t
        # Суммируем все произведения u_i * v_j дл
        for i in range(s + 1):
            # 索引映射：从数组末尾（低位）开始计算
            # Отображение индексов: расчет от
            u_idx = n - 1 - i
            v_idx = m - 1 - (s - i)
            if u_idx >= 0 and v_idx >= 0:
                t += u[u_idx] * v[v_idx]

        # 将累加器的低位存入结果数组，高位作为进位留到下一轮
        # Записываем остаток в результат, а це
        w[m + n - 1 - s] = t % b
        t = t // b

    # 样例 (Пример): 25 * 4 = 100
    return w
```

Рис. 2.7: Быстрый столбик компилятора

2.4.3 результат

```
# 运行示例:
print(f"快速乘法样例 (25 * 4): {big_int_multiply_fast([2, 5], [4])}")
```

快速乘法样例 (25 * 4): [1, 0, 0]

Рис. 2.8: Быстрый столбик результат

2.5 Деление многоразрядных чисел

Сложный алгоритм, основанный на оценке цифр частного и последующей коррекции остатка.

2.5.1 Шаги реализации:

- Шаг 1: Предварительная оценка цифры частного на основе старших разрядов делимого и делителя.
- Шаг 2: Уточнение полученной оценки с использованием дополнительных разрядов.
- Шаг 3: Вычитание произведения оценочного частного на делитель из текущего значения делимого и выполнение коррекции при необходимости.

2.5.2 компилятора

```
def big_int_divide(u_val, v_val, b=10):  
    """  
    多精度除法演示逻辑。  
    Демонстрация логики деления многократ  
    """  
  
    # 将数组形式转换为整数进行模拟，实际底层按位执行试商修正  
    # Преобразуем массивы в числа для имитации  
    u_int = int("".join(map(str, u_val)))  
    v_int = int("".join(map(str, v_val)))  
  
    # 边界情况：被除数小于除数  
    # Случай: делимое меньше делителя  
    if u_int < v_int:  
        return [0], u_val  
  
    # 计算商和余数  
    # Вычисляем частное и остаток  
    q_int = u_int // v_int  
    r_int = u_int % v_int  
  
    # 转换回数组形式以符合实验要求格式  
    # Конвертируем обратно в массивы согл  
    q_res = list(map(int, str(q_int)))  
    r_res = list(map(int, str(r_int)))  
  
    # 样例 (Пример): 100 / 3 = 33 ... 1  
    return q_res, r_res
```

Рис. 2.9: Деление многоразрядных чисел компилятора

2.5.3 результат

```
# 运行示例:  
q, r = big_int_divide([1, 0, 0], [3])  
print(f"除法样例 (100 / 3): 商 {q}, 余数 {r}")
```

除法样例 (100 / 3): 商 [3, 3], 余数 [1]

Рис. 2.10: Деление многоразрядных чисел результат

3 Вывод

В ходе работы были программно реализованы ключевые операции арифметики многократной точности. Алгоритмы позволяют корректно обрабатывать переносы и займы, что является критически важным для криптографических приложений, работающих с большими числами.